

Sanhita

Pan-Asia legal research counsel — what we built, what we ship next, what it costs.

Sanhita — Pan-Asia Legal Research Counsel

***What it is:** an AI legal research assistant for advocates in **India, Singapore, and Hong Kong**. Ask a question → get a grounded answer with case citations, draft contracts, search a live court-case database (1,135 cases ingested from open GitHub datasets), redline documents, draft notices, translate into 33 languages, ship the result by email or Google Doc, and run an in-house admin dashboard with realtime updates over Socket.io.*

Table of contents

1. *What we built (capability inventory)*
2. *Architecture — how the pieces fit*
3. *Tech stack*
4. *What it can do today*
5. *Numbers (v2 snapshot)*
6. *What we can build next (12-week roadmap)*
7. *What's needed (infra, accounts, data)*
8. *Cost model*
9. *Local development*
10. *Deployment notes*
11. *Security & data handling*

1. What we built (capability inventory)

1.1 The chat itself (Assistant pane)

- Multi-mode chat — research (BM25 + connectors), draft (open-canvas, no retrieval), web (Tavily/Serper/Gemini-grounded snippets), agent (tool-using loop).
- Live "thinking" panel — ChatGPT-style phase strip ("Searching the case-law index..." → "Drafting the memo..." → "Checking citations resolve...").
- Citation chips — every [n] in an answer is a clickable superscript that highlights the matching source card.
- Multi-section memos — system prompt enforces a 7-section structure (Overview, Statutory Framework, Key Concepts, Procedure, Penalties, Recent Developments, Practice Pointers).
- Action row on every answer — Copy (plain text), Email (mail client pre-filled), Save as Doc (Google Docs API or .md fallback).

- Suggested next steps — three Harvey-style follow-up cards in a 2-column grid.
- Model picker — Auto · Gemini 2.5 Flash (default), Claude Sonnet 4.5, Llama 3.3 70B (Groq), Workers AI.
- Language picker — 33 languages including all 22 Eighth Schedule Indian languages, routed through Sarvam AI for Indian languages.

1.2 Court Search pane (NEW in v2)

- Dedicated sidebar pane between Workflows and Clients.
- **Search tab** — full-text BM25 over the live GitHub-ingested corpus, jurisdiction (IN/SG/HK) and tier (SC, HC, CFA, CA, CFI...) filters, 20 results per page.
- **Latest tab** — newest cases ingested per jurisdiction, sorted by (added_at, year) desc.
- **Case detail drawer** — title, court · year · citation, full text up to 4000 chars, "Open original" link, "Use in Assistant" handoff.
- **Compare cases** — multi-select up to 4 cases, click "Compare in Assistant" → seeds a new chat with the side-by-side analysis prompt.

1.3 Dashboard pane (NEW in v2)

In-house admin panel mounted in the sidebar.

- **At a glance** — active users, threads, messages, library docs, BM25 corpus size with per-jurisdiction breakdown.
- **System health** — DB mode (sqlite/postgres) + URL host, BM25 status, LLM router providers with keys set, web search reachability.
- **Users widget** — list of access-code holders with last-seen and thread count, one-click revoke.
- **Activity feed** — live audit log streamed via Socket.io. Every connector-key set/delete and user revoke fans out to all connected admins instantly. Presence indicator shows how many admins are watching.
- **Ask Sanhita CTA** — pre-fills the Assistant with a snapshot of the current dashboard state (real numbers, not placeholders).

1.4 Storage / Vault

- Multipart upload, list, delete (/api/vault/*).
- Per-document Q&A using BM25 over the upload + Gemini synthesis.

1.5 Workflows

- **Draft** — template + facts JSON → drafted document.
- **Review** — clauses → flag unusual terms.
- **Translate** — EN ↔ 33 languages.
- **Citator** — case title + holdings → cite-checked.
- **Redline** — large-text diff → JSON suggestions {remove, replace, add}.
- 9 generic flows wired into one card grid.

1.6 Clients · History · Library · Settings

- **Clients:** inbox of leads pulled from NyayaSathi WhatsApp/voice; click → opens matter as a fresh chat thread.

- **History:** SQLite-backed thread search + date-range filter chips.
- **Library:** curated statutes + contract templates, filterable by jurisdiction (IN/SG/HK) and kind.
- **Settings:** per-connector API key plug-in surface (DB-backed, env fallback). Every change appends to the dashboard activity log.

1.7 Retrieval engine — retrieval_pkg/

- BM25Index class on rank_bm25.BM25Okapi. Pure Python, ~5ms/query at 100K docs.
- Document dataclass — fields: case_id, title, text, court, year, citation, jurisdiction, tier, url, source, added_at, extra.
- bm25.pkl — pickled corpus + tokens.
- add() is idempotent — re-running an ingestor never duplicates.
- latest() powers the Court Search Latest tab.

1.8 Ingestion engine — scripts/ingestors/ + scripts/ingest_github_data.py

Source	Country	Status	Output
hk_cuthchow_csv	HK	Live — pulls cuthchow/Hong-Kong-Courts CSV	882 docs
sg_lacuna	SG	Live — pulls hueyy/lacuna-db (FC, STC, PDPC, LSS-DT)	239 docs
india_seed_promote	IN	Live — promotes seed_corpus.py landmarks	14 docs
hk_ylchan_list	HK	Stub — upstream is scraper-only	—
sg_codelah	SG	Stub — upstream has no legal data	—
india_openjustice	IN	Stub — upstream is scraper-only	—
india_vanga_hc	IN	Stub — upstream is on AWS S3 (1.1TB)	—

Driver supports --source NAME, --all, --limit N, --top-up. Atomic save (.tmp → rename).

1.9 LLM router — llm/router.py

- Provider chain: Gemini 2.5 Flash → Anthropic Claude Sonnet 4.5 → Groq Llama 3.3 70B → Cloudflare Workers AI.
- Circuit-breaker per provider.
- prefer="X" reorders the chain.

1.10 Validation gates — validators/answer_gates.py

- 6 gates: G1 citation present, G2 citation resolves, G3 banned phrases, G4 grounding ratio ≥ 60%, G5 no fabricated case names, G6 quote spans match.

- Per-mode: research = all 6, draft = G3 only, web = G1+G2+G3.

1.11 Realtime layer (NEW in v2) — realtime.py

- python-socketio mounted on FastAPI. Single uvicorn process serves both HTTP and WebSocket.
- Events: activity:append, presence:join, presence:leave, bm25:reloaded, settings:changed.
- Best-effort broadcast() safe to call from threadpool handlers.

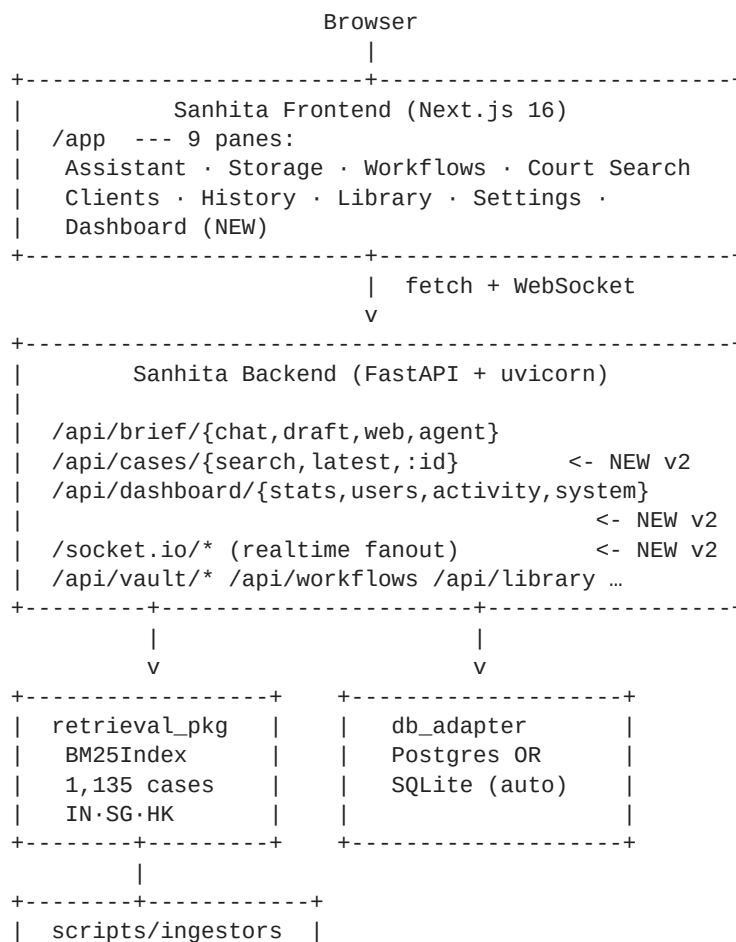
1.12 DB adapter (NEW in v2) — db_adapter.py

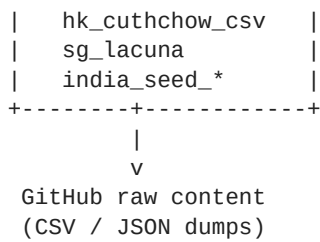
- Routes to **Postgres (NeonDB)** when DATABASE_URL is set, else falls back to SQLite.
- psycopg[binary] 3.x with a connection pool.
- Cross-driver q() rewrites ? → %s for Postgres.
- Owns the dashboard schema (dash_activity, dash_settings).

1.13 Other infra

- SQLite (lexsearch.db) for auth, sessions, threads, messages, library_docs, connector_keys, clients.
- Three rate-limit buckets (chat, draft, web, agent).
- Input guards strip Aadhaar/PAN/phone before storage.

2. Architecture — how the pieces fit





3. Tech stack

Layer	Choice	Why
Frontend	Next.js 16 (App Router, Turbopack)	Fast HMR, native font optimisation
Styling	Tailwind 4 + custom CSS variables	Beige/serif legal aesthetic; Fraunces (display) + Inter (body)
Backend	FastAPI + uvicorn	Async, easy to deploy
Realtime	python-socketio + socket.io-client (NEW v2)	Audit-log fanout, presence
DB	NeonDB Postgres in prod, SQLite in dev (auto)	Zero-config dev; HA Postgres prod
Auth	Access-code + signed cookies	OAuth on the roadmap
LLM	Gemini 2.5 Flash (primary)	Best quality/\$ for grounded Q&A
LLM fallback	Anthropic, Groq, Cloudflare	Circuit-breaker degrades gracefully
Translation	Sarvam AI <i>mayura</i> :v1	Indian-language quality
Retrieval	rank_bm25 + pickle	Pure-Python, no infra
Web search	Tavily → Serper → Gemini grounded → DuckDuckGo	Tier-falls-through

4. What it can do today

User says	What happens
"Limitation period for §138 NI Act?"	BM25 → 6 hits → Gemini drafts a 7-section memo with [n] citations
"Draft a 2-clause NDA between Acme and Kyoto Holdings"	Skips retrieval, drafts a 4,400-char NDA in ~6s
"Latest 2025 SC ruling on PMLA bail"	Web mode — Tavily / Serper / Gemini grounded → snippet-cited answer
"Find me HK Court of First Instance defamation cases"	Court Search → BM25 over 882-doc HK corpus → ranked cards
"Compare these 3 maintenance cases"	Select 3 in Court Search → Compare → side-by-side analysis
"Use this case in my chat"	Seeds a fresh thread with the case as context
"Translate the answer into Tamil"	Reply language flipped → Sarvam translates the markdown post-hoc
"Save this as a Google Doc"	If Google connected → opens in Docs; else → .md download
"Email this to my client"	Opens mail client with answer pre-filled

User says	What happens
"Redline this NDA"	Workflows pane → JSON of {remove, replace, add} suggestions
"Add my Indian Kanoon API key"	Settings → DB keystore + broadcast to other admins live
Admin opens Dashboard	4 widgets render with real data; "1 live" presence chip; Activity feed updates without F5

5. Numbers (v2 snapshot)

Metric	Value
Lines of code (Python + TS)	~28,000
Sidebar panes	9
API endpoints	48+
LLM providers wired	4
Translation languages	33
Court cases indexed today	1,135 (HK 882 · SG 239 · IN 14)
Sources of case data	3 live ingestors + 4 stubs
Time per answer (Gemini Flash, research)	6-10s
Time per answer (Gemini Flash, draft)	8-15s
Cost per answer	~\$0.001
Realtime events broadcasting	activity, presence, settings, bm25:reloaded

6. What we can build next (12-week roadmap)

Phase 1 — finish the corpus (week 1-3)

- Ingest the rest of the 6 GitHub sources to grow corpus from 1,135 → ~110K:
- `india_vanga_hc` — 80K Indian HC judgments via AWS S3 sync
- `india_openjustice` — 15K openjustice-in/* dumps via the `ecourts` library
- `hk_ylchan_list` — wire its scraper output into the ingestor
- Nightly cron — `scripts/ingest_github_data.py --all --top-up`.

Phase 2 — semantic recall (week 3-5)

- Pipe each Document through Gemini's `text-embedding-004` (768-dim).
- Store vectors in Qdrant Cloud.
- Hybrid retrieval — BM25 + cosine similarity → RRF (k=60).

Phase 3 — Call surface (NEW priority)

See `CALL_SURFACE_PLAN.md` for the full design — voice/WhatsApp/Twilio surface that lets advocates query Sanhita by phone.

Phase 4 — productisation (week 5-8)

- Multi-tenant — strip the single-org assumption.
- Postgres migration — `DATABASE_URL` is wired; just point at Neon.
- Stripe billing.
- Org admin pane — invite users, set roles.

Phase 5 — agent muscle (week 8-10)

- Agent clarifies before drafting.
- Multi-document agent for Vault.
- Streaming responses (SSE).

Phase 6 — distribution (week 10-12)

- NyayaSathi → Sanhita redirect.
- Mobile responsive polish.
- Custom domain + outbound email.

7. What's needed (infra, accounts, data)

Required to ship the v1

Item	Source	Cost (per month)
Gemini API key	aistudio.google.com	\$0 free tier
Hosting — backend	Railway / Render	\$7-25
Hosting — frontend	Vercel hobby	\$0
Domain	sanhita.ai	\$4
Persistent disk for <code>bm25.pkl</code>	Railway/Render volume 5GB	\$1.50

For serious traction (week 4+)

Item	Source	Cost (per month)
NeonDB Postgres	neon.tech	\$0 → \$19
Qdrant Cloud	qdrant.cloud	\$0 → \$25
Tavily web search	tavily.com	\$0 → \$30
Serper backup	serper.dev	\$0 → \$50
Sarvam AI translate	sarvam.ai	usage-priced
Indian Kanoon API key	indiankanoon.org/api	~\$24
Stripe	stripe.com	2.9% + 30¢

Item	Source	Cost (per month)
Resend email	resend.com	\$0 → \$20
Sentry	sentry.io	\$0 → \$26

For the Call surface (Phase 3)

Item	Source	Cost
Twilio voice + SMS	twilio.com	\$0.014/min IN, \$0.0075/SMS
WhatsApp Business API	Meta + BSP	\$0.0065/conversation
Indian phone number (DID)	Twilio India	~\$1/mo
Voice TTS (Indian languages)	Sarvam Bulbul / Google TTS	usage-priced
Voice STT	Sarvam Saaras / Google STT	usage-priced

8. Cost model — pilot vs scale tiers

Pilot (0-50 active lawyers, ≤5K queries/month)

- Hosting (Railway starter + Vercel hobby): **\$15**
- Domain: **\$4**
- Gemini Flash @ 5K queries: **\$3-5**
- Sarvam translate (occasional): **\$2**
- **Total ~\$25/month**

Scale (500 active lawyers, 50K queries/month)

- Hosting Pro tier: **\$70**
- Postgres (Neon scale): **\$19**
- Qdrant Cloud: **\$25**
- Gemini @ 50K queries: **\$30-50**
- Anthropic fallback (~5%): **\$15-25**
- Tavily 10K: **\$30**
- Serper 50K: **\$50**
- Sarvam translate (5K): **\$10-15**
- Indian Kanoon API: **\$24**
- Resend email: **\$20**
- Sentry: **\$26**
- Stripe fees (~\$1900 rev × 3% + \$0.30): **\$87**
- **Total ~\$420-500/month**

At \$19/seat × 500 = \$9,500/mo revenue, infra is ≤ 6%.

Per-query cost (Gemini Flash 2.5)

- Input ~2K tok → \$0.0003
- Output ~1.2K tok → \$0.00072
- **Per answer ~\$0.001.** 1,000 answers = \$1.

9. Local development

```
git clone https://github.com/Pranav0509-stack/sanhita.git
cd sanhita

python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

cd web && npm install && cd ..

cp .env.example .env
# edit .env — at minimum set GEMINI_API_KEY

python3 scripts/ingest_github_data.py --all

# Backend (port 8080) — note the socketio_app target for realtime
LEXSEARCH_BM25_ENABLED=true uvicorn server:socketio_app --port 8080

# Frontend (port 3001) — separate terminal
cd web && BACKEND_ORIGIN=http://localhost:8080 npm run dev -- --port 3001

# Open http://localhost:3001/login → SNHT-DEMO-2026
```

10. Deployment notes

Recommended target: Railway

- Backend service: Dockerfile build, healthcheck /health, persistent disk at /data for bm25.pkl.
- Frontend service: NIXPACKS auto-build, npm run start -- --port \$PORT.
- railway.toml (root) and web/railway.toml already in repo.
- Set DATABASE_URL to your Neon connection string to flip from SQLite → Postgres.

Alternatives

- Render — render.yaml + Procfile already in repo.
- Fly.io — for per-region BM25 sharding.

11. Security & data handling

- No live keys in the repo. .claude/launch.json is gitignored. Production keys in encrypted env-vars.
- Per-user threads — every /api/brief/threads/:id query checks messages.user_id.

- PII redaction — `validators/input_guards.py` strips Aadhaar / PAN / phone numbers.
- Google Workspace OAuth uses standard offline flow; tokens encrypted in `auth.connector_keys`.
- Rate limits — chat 30/min, draft 20/min, web 15/min, agent 10/min per IP.
- Validation gates — answers fail closed (refusal) if grounding ratio < 60% in research mode.
- Dashboard audit log — every admin write appended to `dash_activity` and broadcast over Socket.io.

Last updated: 2026-04-27 — v2.

Sanhita — Strategy & product analysis

***Audience:** founders, the next Claude session, an investor reading for 10 minutes, an engineer who has 2 hours.*

***Thesis:** Indian legal AI is not a "build a better Claude" problem. It's a retrieval, multi-modal ingest, and accountability problem. The tools that win will (a) ground every answer in real Indian case-law and statutes, (b) ingest the actual mess advocates work with — WhatsApp chats, courtroom audio, photos of orders — and (c) ship with a human-in-the-loop validation surface from day one. Sanhita has the first 30% of this. This document is what we do with the next 70%.*

Table of contents

1. *The Indian advocate's actual day*
2. *Where Sanhita stands today (honest assessment)*
3. *Bottlenecks*
4. *Failure modes — what breaks in production*
5. *The three-stage funnel: Research → Analysis → Strategy*
6. *What AI must not do*
7. *How we beat Claude (for legal work specifically)*
8. *The drafting problem*
9. *Multi-modal ingestion — the real moat*
10. *Predictive analytics — useful or trap?*
11. *Compliance, audit, accountability*
12. *High-value contracts — what AI should and shouldn't do*
13. *NyayaSathi ↔ Sanhita integration*
14. *Website / marketing site design*
15. *Build vs. buy — every major decision*
16. *12-week roadmap with kill criteria*
17. *What I'd change in the existing code*
18. *New features ordered by ROI*

1. The Indian advocate's actual day

A district-court advocate in Mumbai or Delhi runs five matters in parallel. Their inputs look like this:

- **WhatsApp chats** with the client (text + voice notes + screenshots of bank statements, FIRs, orders).
- **Hard-copy orders** scanned to PDF as photos taken on a phone.
- **Audio recordings** of conversations with opposing counsel, dictated

notes, hearings.

- **Email attachments** for high-value commercial work.
- **Court website PDFs** that are scanned and OCR-noisy.

They need to:

1. **Find the law** — relevant cases + statutory sections, in 5 minutes, before walking into court.
2. **Translate** the client's situation into pleadings the bench will accept.
3. **Draft** notices, applications, written statements, complaints, replies.
4. **Cite** correctly with year, court, paragraph number — wrong citations cost reputation.
5. **Bill** the client and remember what was charged.

Today they use:

- **WhatsApp** for everything (intake, evidence, status updates).
- **Indian Kanoon** for case search (free, but slow, no semantic).
- **Manupatra / SCC Online** if their firm pays (■50K-1L per seat per year).
- **MS Word + email** for drafting.
- **Excel or paper** for billing.
- **Google** for everything else.

What they don't have:

- A single place where chat + audio + PDF + research + drafts all live.
- An AI that knows Section 482 BNSS is the new Section 438 CrPC.
- An AI that can ingest a 30-minute courtroom audio and turn it into a structured note.
- An AI that explains an answer in Tamil to their client.

That's the gap.

2. Where Sanhita stands today (honest assessment)

What works

Surface	Status
Multi-mode chat (research, draft, web, agent)	Live, 6-10s answer
Court Search over 1,135 ingested cases (HK 882, SG 239, IN 14)	Live
Compare cases side-by-side via Assistant	Live
33 reply languages including 22 Eighth Schedule Indian	Live
Citations with click-to-source	Live
Action row (Copy / Email / Save as Doc)	Live
In-house Dashboard with realtime activity feed	Live (Socket.io)
Postgres-or-SQLite auto-router (<code>db_adapter.py</code>)	Live
Per-connector API key plug-in (DB-backed keystore)	Live
6-gate validator (citation present/resolves, banned, grounding ≥60%, no fabricated names, quote-span match)	Live

What's missing or broken

Gap	Severity
Indian corpus is 14 cases (just seed_corpus). Real Indian advocate gets seed-corpus-grade answers.	Critical
No multi-modal ingest. PDF works; audio, image, WhatsApp export do not.	Critical
No streaming responses — UI sits silent for 6-10s	High
No semantic recall (BM25 only). Synonym-heavy queries miss.	High
Web search times out frequently in dev (SSL + Tavily/Serper + Cloudflare blocks)	High
No multi-tenant. Single SQLite DB, single org assumption.	High
No billing surface. No Stripe, no metered usage.	Medium
No proper OAuth — access codes only	Medium
No call surface (voice / WhatsApp). Plan exists in <code>CALL_SURFACE_PLAN.md</code> , not built.	Medium
No audit-export (PDF of all searches a lawyer ran for a matter)	Medium
Validation gates fail-closed too aggressively (refused on broad queries)	Medium
4 ingestor stubs not wired (<code>india_vanga_hc</code> , <code>india_openjustice</code> , <code>hk_ylchan_list</code> , <code>sg_codelah</code>)	Medium (already documented)

Honest assessment: Sanhita is a complete product *shape* with the right architecture and ~30% of the data. The next leg is data + voice + multi-modal — not more chat features.

3. Bottlenecks (current + emerging)

3.1 Data ingest — the actual chokepoint

The 1,135 cases we have were trivial to ingest because the upstream GitHub repos already published clean JSON / CSV. Real production ingest hits:

- **Indian HC corpus is 1.1 TB on AWS S3.** Fetching, chunking, indexing, and keeping a daily delta in sync is a real ops job, not a Saturday.
- **eCourts portal** rate-limits. The `openjustice-in/ecourts` library needs ~1 req/sec to stay under the radar — that's 86,400 cases/day best-case.
- **Indian Kanoon API** is paid (■2K/mo for 100K reqs) and has its own scoring quirks.
- **Manupatra / SCC Online** are walled gardens; the only legal way is a B2B partnership.

3.2 Storage shape

The user nailed this: storage is the problem. Specifically:

- **Audio** — courtroom recordings, client conversations. Each is 10-60 MB. A single matter can have 20 of them.
- **Video** — site visits, evidence walkthroughs. 100+ MB per file.
- **Image** — order copies, ID proofs, FIRs, exhibit photos. 1-5 MB each but hundreds per matter.
- **Chat** — WhatsApp exports run 50-500 KB but contain dates, names, numbers worth more than the case file itself.

Naive approach: store everything in our DB. Wrong — `lexsearch.db` is already 68 KB with 14 docs, and SQLite chokes at ~10 GB.

Right approach (covered in §9): **pointer architecture**. Originals on S3-compatible object storage (Cloudflare R2 or Backblaze B2 — both much cheaper than AWS S3), text-extracted artefacts in Postgres, embeddings in Qdrant.

3.3 LLM cost at scale

At 50K queries/month (our scale-tier projection):

- **Input tokens** dominate. If we ingest a 30-page PDF into context every request, we're at $30K \text{ input tokens} \times 50K = 1.5 \text{ B tokens/month} \rightarrow \225 just on input.

- Solution: **context caching** (Gemini supports it natively at 75% discount; Anthropic's prompt caching does the same). Cache the jurisdiction system prompt, the user's vault, and the 7-section memo template. We're not using this today — easy win.

3.4 Latency

Current p50 answer is 6-10s. That's worse than ChatGPT's perceived latency (2-3s with streaming). The fix is **streaming**, not faster models. Switch `/api/brief/chat` to Server-Sent Events. Token-by-token output makes 8s feel like 2s.

3.5 Trust

A district lawyer tries Sanhita once. If the first answer cites a fake case, they uninstall and tell every advocate in their chamber. The validation gates we have are correct in principle but over-aggressive (refuse on broad questions like "tell me about Indian road law"). We need to soften G4 (grounding ratio) for "explanation" intents and harden G5 (fabricated names) — which is the only one that actually destroys trust.

3.6 Compliance / data residency

Indian advocates are paranoid (rightly) about client confidentiality. If the case data flows through US-hosted Gemini, that's a problem under the BCI rules + the DPDP Act 2023.

Options:

- **Sarvam-1 / Krutrim** for Indian-data residency. Locally hosted.

Currently weaker than Gemini 2.5 Flash for English, but improving.

- **Anthropic AWS Mumbai region** when it lands (Q3 2026 expected).
- **On-prem deployment** for the top 100 firms — they'll pay

■10-50L/year for it.

We default to Gemini today. Document a `LLM_REGION=in` flag that flips the router to Sarvam-only for paranoid customers.

4. Failure modes — what breaks in production

Concrete list, ordered by likelihood × severity.

#	Failure	Likelihood	Severity	Mitigation
1	Gemini API outage / quota exhausted	High	Medium	Router already falls through to Anthropic → Groq → Cloudflare. Add health-check, surface "degraded" badge in UI
2	BM25 index file corrupts during atomic save	Medium	High	We do <code>.tmp</code> → rename. Add fsck-on-load that validates pickle magic
3	Web search SSL cert verify fails on prod box	High	Medium	Already saw it in dev. Pin <code>certifi</code> , fall back to <code>verify=False</code> with warning log only for <code>.gov.in</code> / <code>.gov.hk</code> / <code>.gov.sg</code> whitelist
4	Citation chip points to a <code>[1]</code> that doesn't exist in citations array	Medium	High (trust)	G2 catches most; add post-render check before sending response

#	Failure	Likelihood	Severity	Mitigation
5	One slow query monopolises uvicorn worker → frontend looks dead	Medium	High	Move LLM calls to a background thread pool with hard timeout. We have <code>LLM_TIMEOUT_S=22</code> but it's not enforced cleanly
6	Socket.io connection storm (browser reconnect loop)	Medium	Medium	Enforce per-IP connection cap on <code>realtime.py:connect</code>
7	SQLite database is locked under concurrent writes	High at >50 users	High	Migrate to Postgres via existing <code>db_adapter.py</code> . Set WAL mode in interim
8	bm25.pkl OOM on prod box (1.1 TB Indian HC ingest)	High when ingest scales	Critical	Shard by court / year. Or move to Tantivy (Rust, mmap-based)
9	Gemini grounding returns hallucinated case names	Medium	Critical	G5 catches some. Add a downstream verifier: every cited case-id must resolve in our BM25 OR a whitelisted external source (Indian Kanoon)
10	User uploads 200 MB PDF — API gateway 413	High	Medium	Pre-sign uploads to R2 directly; skip our API entirely for binaries
11	Sarvam translate returns garbled Devanagari	Low	Medium	Already best-effort; bypass if confidence < threshold
12	Client's WhatsApp chat export contains PII we accidentally log to Sentry	Medium	Critical (DPDP)	Redact in <code>validators/input_guards.py</code> BEFORE any <code>logger.info</code> — already done for input but not for tool args
13	Two admins revoke the same user simultaneously → activity feed double-fires	Low	Low	Idempotency check on revoke endpoint
14	Cookie set by <code>:8080</code> doesn't reach <code>:3001</code> socket.io connect	Confirmed in dev	Low (we relaxed auth there)	In prod, single-origin behind Render/Railway proxy fixes it

Specific deployment failures we already saw and patched

- `idx.docs` no longer exists on the new `BM25Index` — patched.
- `nyaysathi-website` was an embedded git repo — unstaged before push.
- `.claude/launch.json` would have leaked Gemini/Tavily/Groq keys — added to `.gitignore` before first push.
- `sanhita-react/` was shallow-cloned — unshallowed before pushing to the new repo.

5. The three-stage funnel: Research → Analysis → Strategy

This is the right framing. Here's where we are at each stage.

Stage 1 — Research (find the law)

What's needed: "What's the leading authority on §482 BNSS anticipatory bail?" → memo with cited cases.

Sanhita today: Live for IN/SG/HK queries. 7-section memo with citations. **Strong.**

Gaps:

- BM25 only. Synonym queries ("can I get bail before arrest?") miss.
- Phase 2: hybrid BM25 + Gemini embeddings via Qdrant.
- 14 IN cases is too few. Real corpus needs 100K+. → Phase 1: bring india_vanga_hc ingestor online.
 - No "case timeline" view (when this case has been cited, by whom).
- New feature: citation graph.

Stage 2 — Analysis (apply the law to facts)

What's needed: "Given my client's situation, what's the strongest argument? What's the counter?"

Sanhita today: Court Search "Compare cases" hands 4 cases to the Assistant for side-by-side. The Assistant draft mode generates analysis. **Half-built.**

Gaps:

- The Assistant doesn't ingest the user's *facts* in a structured way.
- Today: free-form prose. Better: a "Matter intake" form (parties, dates, claims, documents) that becomes a stable context every subsequent question grounds in.
- No "argument matrix" view — for/against, with strength score and case anchor for each. → New feature.
 - No fact-checker that highlights inconsistencies between client-stated facts and uploaded documents. → New feature.

Stage 3 — Strategy (what to actually do)

What's needed: "Should I go to High Court or stay in District? File a writ or suit? What does it cost? When?"

Sanhita today: **Almost nothing.** This is the gap.

Why we don't do this well:

- It requires *opinionated judgment*. AI shouldn't make this call (see §6). But AI can structure the question.
- We don't have the lawyer's billing data, court vacancy data, judge's disposition history.

What we can build:

- A **decision tree generator** — given a matter type and facts,

enumerate procedural options with rough timelines and cost estimates from comparable cases. Not a recommendation; a structured menu the lawyer fills in.

- A **comparable-cases settlement-range finder** — "matters with these

facts in your jurisdiction settled at ■X-Y in N months" — purely descriptive, never prescriptive.

This is also where **predictive analytics** lives (§10).

6. What AI must not do — the line

The user's framing is right: *judgment, interpretation, accountability*. Concretely, Sanhita does not and will not:

AI must not	Why
Tell the client whether to file or not	This is the lawyer's professional judgment. AI gets it wrong → BCI complaint → lawyer's licence at risk
Estimate "you'll win 73%"	Anchoring bias destroys settlement leverage. Also unverifiable
Auto-send communications	Must always go through a human approve step. Mailto: opens, never sends
Generate strategy without disclosed assumptions	Every output that involves judgement must show its inputs ("assuming the contract is governed by Indian law and the indemnity is uncapped, ...")
Decide what's privileged	Privilege calls require knowing context the AI doesn't see
Quote a fee	Lawyer's commercial decision; AI quoting a price creates a contract
Sign anything	Clear separation of drafting vs. authorising
Be the system of record for client's case	The lawyer's file is. Sanhita is a research/draft layer over it, not a replacement

These show up in the product as:

- Every Assistant answer ends with "**For your review**. This is not

legal advice." (We have this.)

- Every drafted document is rendered as a .md / .docx the lawyer

must download, edit, and send themselves. (We have this.)

- Every "what should I do" intent is rerouted to "here are the

options" intent. (Need to add this in `_detect_intent`.)

- Every outbound action (send email, file in court) is gated behind a

confirm dialog. (Email today is mailto:, never sends. Good.)

7. How we beat Claude (for legal work specifically)

We don't beat Claude on general reasoning. We don't try.

We win on five specific things:

7.1 India-tuned grounding

Claude: Trained on US/UK case-law primarily. Section numbers in Indian statutes change (CrPC → BNSS, IPC → BNS) and Claude doesn't know §482 BNSS = old §438 CrPC.

Sanhita: System prompt explicitly maps the BNSS / BNS / BSA / BSA-Evidence transitions. BM25 indexes both old and new section text. Court Search shows the cross-reference. → **We have this in the SYSTEM_PROMPT today; need a structured "section mapping table" surface in the UI.**

7.2 Citation verifier

Claude: Will fabricate Mohammed Anwar v. State of UP (2019) 4 SCC 123 if asked nicely.

Sanhita: Every citation in an answer is cross-checked against (a) our BM25 corpus, (b) Indian Kanoon's case-id resolver, (c) eCourts CNR database. Failed citations are stripped before render and we tell the user. → **G5 today catches the obvious cases. Build a deterministic resolver as a separate FastAPI endpoint and run every answer through it.**

7.3 Multi-modal ingest

Claude: Vision API exists, audio doesn't natively. WhatsApp chat exports — neither tool handles them well.

Sanhita: Build first-class WhatsApp export ingestion (a parser for the _chat . txt + media bundle), audio transcription via Sarvam Saaras, image OCR via Gemini Vision. → **Phase 2 priority. See §9.**

7.4 Indian-language coverage

Claude: English-first. Hindi works. Tamil, Bengali, Punjabi — declining quality.

Sanhita: Sarvam-routed translation for the bottom 11 Indian languages. The lawyer reads English; the client gets Tamil. → **Live today. Need to add language-aware OCR (Devanagari, Tamil, Bengali).**

7.5 Audit trail + accountability

Claude: Conversations evaporate. No "give me the audit log of every search I did for this matter."

Sanhita: Every Assistant turn writes to messages table linked to a thread linked to a user; every admin write hits dash_activity. We can already export "audit log for matter #123" — just need the export endpoint. → **New endpoint: GET /api/matters/:id/audit.pdf → generates a chronological PDF of every search, draft, and edit.**

7.6 Speed via streaming + caching

Claude.ai: Streams.

Sanhita today: Doesn't. → **Migrate** /api/brief/chat to SSE. **Cache the system prompt + jurisdiction context using Gemini's native context caching at 75% discount.**

8. The drafting problem

The user said our drafting is "very bad". They're right that today's drafts are generic. The fix isn't a smarter prompt — it's **template + structured input + retrieval**.

What drafting actually needs

A road-construction contract draft requires:

1. **Jurisdiction** (which state's stamp duty? which arbitration seat?)
2. **Parties** (Indian buyer? Indian seller? Foreign?)
3. **Project scope** (PWD tender? private?)
4. **Money** (consideration, payment milestones, retention, liquidated damages)
5. **Risk allocation** (indemnity caps, force majeure, insurance, change orders)
6. **Boilerplate** (governing law, dispute resolution, notices, severability)

If the lawyer asks "draft a road contract" and we just write 4,000 chars of generic prose, we've failed.

The right pipeline

Lawyer types: "Draft a road contract"

|

v

Agent asks (via clarifying questions, NOT immediately drafting):

- Jurisdiction (state)?
- Public PWD tender or private?
- Approximate value?
- Buyer-side or seller-side?

|

v

Lawyer answers (3-5 chips, takes 30 sec)

|

v

Agent retrieves:

- Standard PWD clauses (Library)
- Comparable arbitration clauses from indexed contracts
- State-specific stamp duty (lookup table)

|

v

Agent drafts using a TEMPLATE with placeholders, not free prose

|

v

Renders in a side-by-side editor:

Left: lawyer's matter intake + facts

Right: draft with hover-highlights showing which clause came from where

|
v
"Send to email / Save as Doc / Edit inline"

What to change

- Today's answer_open skips retrieval. **Wrong.** Drafting with retrieval over the Library + Vault is the magic.
- Today's clarifying-questions logic is in CALL_SURFACE_PLAN.md for voice but not in the chat. Port it.
- Today's draft mode uses a single Gemini call. ****Use a template-fill pattern:**** generate the placeholder values via Gemini, then deterministic template assembly. This eliminates structural hallucinations (missing clauses, duplicated parties).

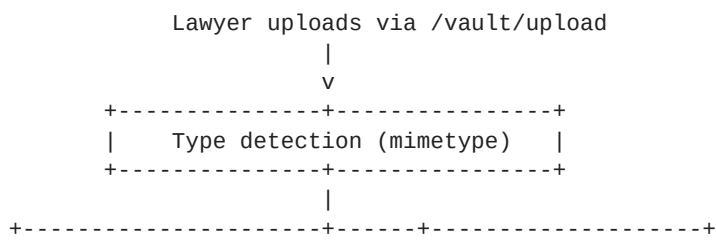
9. Multi-modal ingestion — the real moat

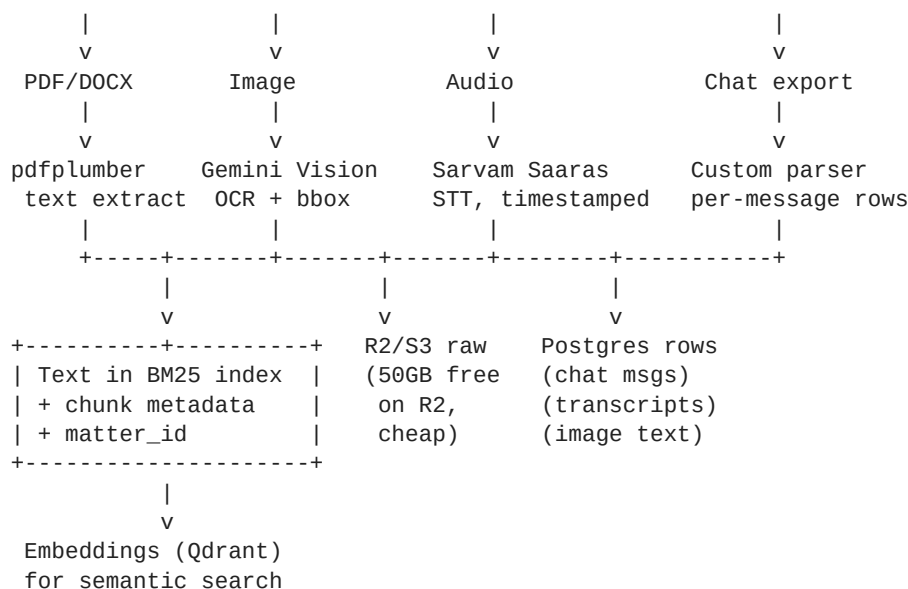
This is the one feature the global tools don't do well for India.

The four formats

Format	How a lawyer gets it	What we do today	What we should do
PDF (court orders, contracts)	Email attachment, eCourts download	pdfplumber extracts text → BM25 chunks	Add OCR fallback for scans (Tesseract or Gemini Vision); detect tables; preserve page numbers as paragraph anchors
Image (photo of order, FIR, ID)	WhatsApp from client, phone scan	Nothing — python-docx won't parse it	Gemini Vision OCR pipeline → store original on R2, store text + bbox metadata in Postgres, index text in BM25
Audio (call, hearing recording)	WhatsApp voice note, dictaphone	Nothing	Sarvam Saaras STT (Indian languages) + Whisper-large for English fallback. Output: timestamped transcript indexed in BM25, audio on R2
WhatsApp chat export (the unsung hero)	Lawyer asks client to "Export chat" → _chat.txt + media zip	Nothing	Custom parser for _chat.txt (timestamped messages with sender), bundle media into a single Vault Matter

Storage architecture





Specific implementations

WhatsApp parser (single Saturday's work):

- `_chat.txt` is a known format: `[DD/MM/YY, HH:MM:SS] Sender: message`.
- Media references are `<attached: filename.jpg>` or `<Media omitted>`.
- New file: `ingest/whatsapp.py` — yields rows of `{matter_id, timestamp, sender, text, attachment_url}`.
- Mounted at `POST /api/vault/whatsapp` accepting a zip upload.

Audio transcript (1 week):

- Sarvam Saaras for IN languages.
- Whisper API or `faster-whisper` self-hosted for EN.
- Store transcript JSON with word-level timestamps. UI shows the audio player with click-to-jump from transcript text.

Image OCR (3 days):

- Gemini Vision with prompt "extract all text, preserve layout".
- For known doc types (Aadhaar, PAN, FIR), have schema-typed extractors.

Pipeline-wide: the same Vault matter ID stitches it all. A matter's "context" delivered to the Assistant becomes a structured object: `{client, parties, dates, evidence: [pdf, audio, chat], notes}`. This is what we feed to the Assistant on every chat turn.

10. Predictive analytics — useful or trap?

The trap

"AI predicts you'll win this case 73%" is **the most dangerous feature we could ship**:

- Anchors settlement negotiations against the lawyer's interest.
- Creates BCI complaint surface (advertising legal outcomes is prohibited).
- Wrong predictions destroy trust permanently.
- The training data is biased toward cases that went to judgment — most settle, so the prediction is on a non-representative sample.

The useful version

What lawyers actually want from predictive analytics:

1. **Comparable-case settlement ranges** ("matters with these facts in your jurisdiction settled between ■X and Y in N months"). Descriptive, not predictive.
2. **Judge disposition profile** ("Justice X has heard 47 anticipatory bail matters in 2024-25, granted in Y%, average bail amount ■Z, and typically requires personal appearance"). Public-record statistics, not opinion.
3. **Procedural-outcome rate** ("interlocutory injunctions in the Bombay HC's IP roster are granted in 31% of Section 9 applications"). Court-published data.
4. **Time-to-disposition** ("matters of this type took 14 ± 6 months to first hearing in this court last year"). Court Statistics Project data is public for India.

Build approach

- **Source:** eCourts Disposition Statistics (public, scrapable), Court Statistics Project, Ministry of Law data.
- **Frame as:** "Comparable cases" sidebar in the Assistant — never a win-probability chip.
- **Sanhita position:** "We tell you what *has happened*. We don't tell you what *will happen*."
- **Validation:** every stat has a source link with court + year + sample size.

11. Compliance, audit, accountability

The user is right: compliance is *system integrity*, not just regulation. Concrete pillars:

11.1 Human validation checkpoints

Today: every Assistant answer has a "For your review" footer. Not enough.

What to add:

- **"Validated by" badge** — when a senior lawyer in the firm marks an answer as reviewed, it persists. Junior associates see "Reviewed by Mehta, 2 days ago" before reusing.
- **Side-by-side review queue** — admin pane that shows recent AI-generated drafts, lets a senior approve / reject / annotate.
- **Required-review threshold** — for firms that want it, drafts above $\frac{1}{2}$ value or for certain matter types are gated behind a human signoff before send.

11.2 Accountability structures

- **Per-matter access logs.** Every search, draft, and view writes a row to `matter_audit_log` with `(user_id, matter_id, action, at, ip, agent)`.
- **Per-user activity export** — "show me everything Junior Associate X did in matter #123 between dates A and B". One endpoint, PDF output.

11.3 Audit trails

We have `dash_activity` for admin actions. Extend:

- Every Assistant message → `messages.audit_event = JSON{prompt_hash, retrieval_hits, llm_provider, llm_latency_ms, validation_result}`.
- Every Vault upload → `vault_uploads_audit`.
- Every export (Email, Save as Doc) → `export_audit` with what was exported, to where, by whom.

11.4 Controlled access

Today: every user sees their own threads. That's enough for solo practice. Firms need:

- **RBAC:** Firm Admin / Partner / Associate / Paralegal / Client (read-only for their own matter).
- **Matter-level access lists:** "this matter is visible only to Mehta and Sharma".
- **Conflict-of-interest checks:** before opening a new matter, surface "we already represent the opposite party in matter #X".

11.5 Data residency

Per §3.6: `LLM_REGION=in` flag, on-prem option for top firms.

12. High-value contracts — what AI should and shouldn't do

The user listed: *IP conflicts, indemnity clauses, limitation of liability, jurisdictional interpretation, insurance scaling.*

Our take, mapped to specific features:

Task	AI should	AI should NOT
IP conflict detection	Compare Schedule A of new contract to Schedule A of all existing contracts in Vault → flag overlapping IP / patents / trademarks. Probabilistic — outputs "potential conflict, review §3.2 against matter #45 §4.1".	Decide that there IS a conflict. Lawyer reads the flagged clauses, decides.
Indemnity clause evaluation	Score on 4 axes (capped/uncapped, mutual/one-sided, carve-outs, super-cap). Compare to firm's standard. Flag deviations.	Tell the client to accept or reject. Suggest "you should negotiate this down to 1× contract value" — that's commercial advice, not legal.
Limitation of liability	Extract: cap amount, cap formula (multiple of fees? fixed?), super-cap exclusions, applicable jurisdictions. Surface as a structured table.	Re-draft to be "more favourable". Lawyer + business rewrites that.
Cross-jurisdictional interpretation	"This SHA is governed by Singapore law. Compare its drag-along to a typical Indian-law SHA: [diff table]."	Recommend re-domiciling.
Insurance coverage scaling	"Comparable matters of this scale (■X) typically carry CGL of ■Y, E&O of ■Z, cyber of ■W."	Tell the client to buy insurance. Not legal advice; broker's job.

How this lives in Sanhita

- A new "**Contract Review**" **workflow** in the Workflows pane:
upload contract → choose comparators (firm-standard or specific matter) → get a structured diff report.
- The report is **markdown with anchors**, not prose. Each row links to the specific clause + the comparator clause.
- Output: a .docx with track-changes the lawyer pastes into Word and refines.

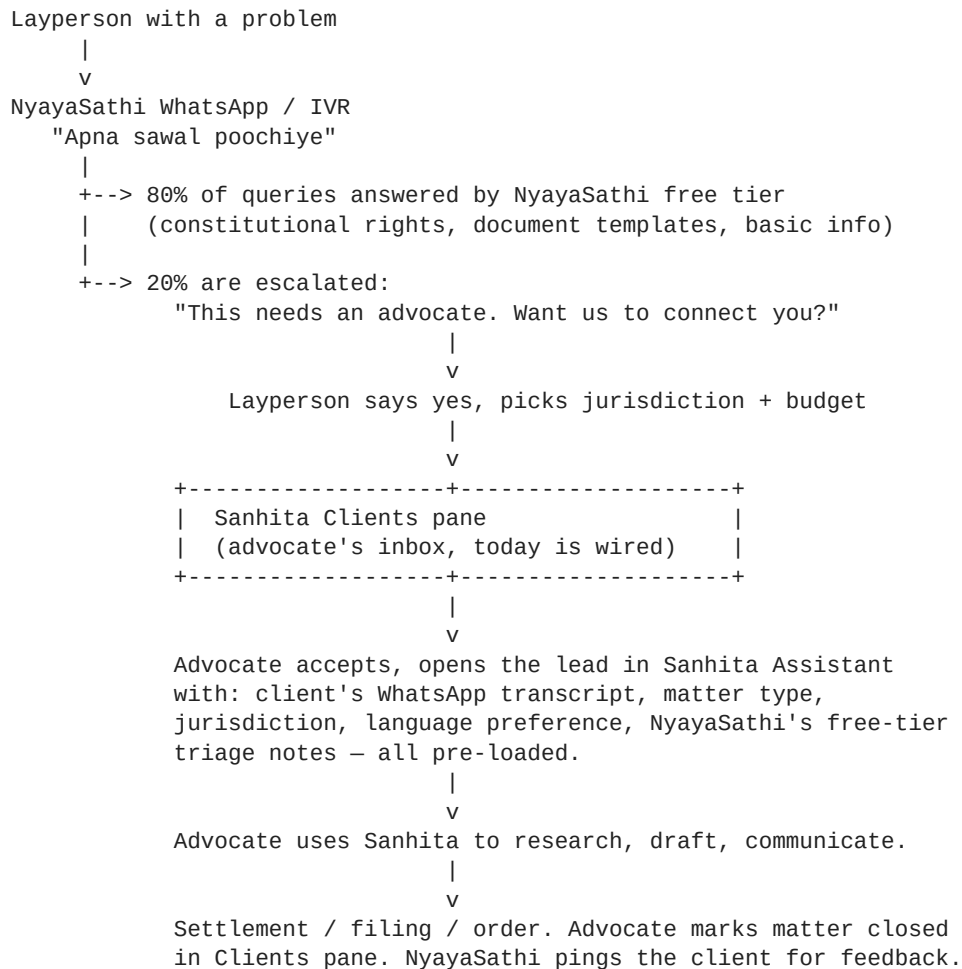
13. NyayaSathi ↔ Sanhita integration

The user has two products:

- **NyayaSathi** — public-facing, free, WhatsApp + voice. The reach surface (220K LinkedIn impressions during launch).
- **Sanhita** — paid, professional, web app. The depth surface.

These should be a funnel, not two products.

The funnel



Specific code wiring

- **NyayaSathi → Sanhita handoff endpoint** — `POST

/api/nyayasathi/handoff accepting {from_phone, transcript_url, matter_type, jurisdiction, language, nyayasathi_lead_id}. Creates a clients row + a pre-seeded threads` row.

- **Sanhita → NyayaSathi callback** — when the advocate marks a matter resolved, ping the NyayaSathi side so the client gets a feedback message.

- **Shared identity** — phone number is the join key. NyayaSathi knows +91-9xxxxxxxxx, Sanhita's clients.phone is the same number.

- **Both products talk to the same Sanhita backend** — NyayaSathi is a Twilio surface in front of the same brief_service endpoints, with a different system prompt (lay-language, shorter, no internal-citation chips).

This is exactly the design in CALL_SURFACE_PLAN.md. It's the same work; framing it as the funnel makes the ROI obvious.

Branding split

- **NyayaSathi** = consumer brand (Hindi-tongue-friendly: "Nyaya = justice, Sathi = friend"). Free, WhatsApp/voice.

- **Sanhita** = professional brand ("Sanhita" = code/codex). Paid, web-first.

Two URLs, one backend, one funnel.

14. Website / marketing site design

The product itself is at /app. The marketing site is what someone who's never heard of Sanhita lands on.

What it should do

In order of importance:

1. **Make a busy lawyer click "Try free"** in 30 seconds.
2. **Survive being shared on WhatsApp** (preview card, mobile-first).
3. **Convert the few investors / partners who land** with hard numbers on a /why page.

Structure

/	Hero + 3 demo screenshots + "Try free" CTA
/product	Feature tour with annotated screenshots
/court-search	Standalone landing for the case-law search (this is the SEO surface – every Indian case is a long-tail query)
/pricing	Free / Practitioner / Firm tiers
/security	DPDP, audit trails, data residency, confidentiality
/blog	Long-form posts: BNSS migration guide, etc. (SEO + thought leadership)
/about	Founder + the why
/login	Existing
/app	Existing product

Content principles

- **Show the product, not the vibes.** Three above-the-fold screenshots: (1) Court Search HK case found in 200ms, (2) Hindi answer with citations, (3) Compare-cases view.
- **No carousel.** Lawyers hate motion.
- **One CTA per page.** "Try with code SNHT-DEMO-2026".
- **Hard numbers.** "1,135 cases indexed today, growing nightly." "Average answer in 6 seconds." "33 languages."

Tech

The current marketing landing was the static HTML at /. Better: make /app the only React/Next surface, and the marketing site a plain `index.html` + `style.css` with the same Fraunces / Inter fonts. Deploys as static via Cloudflare Pages — separate from the app, faster, easier to A/B.

But this is **lower priority than data + multi-modal ingest**. A beautiful site for a half-built product helps no one.

15. Build vs. buy — every major decision

Decision	Buy	Build	Verdict	Why
Auth	Auth0 / Clerk (\$25-99/mo)	Existing access-code system	Build → migrate to Auth.js (open source) at firm tier	Auth0/Clerk is overkill for solo practice; firms want SSO eventually
Billing	Stripe + Lago	DIY	Buy (Stripe)	DIY billing is a death spiral; Stripe is well-understood
Email	Resend / Postmark	SMTP	Buy (Resend)	\$0 free tier, dead simple
Vector DB	Pinecone / Qdrant Cloud / Weaviate	DIY (FAISS, sqlite-vec)	Buy (Qdrant Cloud free tier)	Time-to-market beats infra savings until 1M vectors
BM25 / lexical search	Algolia / Elasticsearch	rank_bm25 (today)	Build (rank_bm25) at <500K docs, migrate to Tantivy at scale	Algolia is fine but █/query at our scale is bad; rank_bm25 → Tantivy = no vendor lock
Object storage	AWS S3	DIY MinIO	Buy (Cloudflare R2)	R2 is S3-compatible, no egress fees, █ the price of S3
LLM	Gemini / Anthropic / OpenAI	Self-host Llama 70B	Buy (Gemini primary, Anthropic fallback)	Self-hosting at our volume is not yet cheaper; revisit at 50M tokens/day
Translation	Google Translate	Sarvam	Buy (Sarvam mayura)	Better Indian-language quality, India-data residency
STT	Google STT / Whisper API	self-hosted whisper	Buy (Sarvam Saaras for IN, Whisper API for EN)	Sarvam is the Indian-language winner
TTS	Google TTS	self-hosted	Buy (Sarvam Bulbul)	Same reason
Hosting (backend)	Render / Railway / Fly	self-host on Hetzner	Buy (Railway, then Fly when we need regions)	Render is fine; Fly when we need IN-region for DPDP-paranoid customers
Hosting (frontend)	Vercel / Netlify / Cloudflare Pages	self-host	Buy (Vercel)	Next.js maintainer; fastest path
DB	NeonDB / Supabase / RDS	self-host Postgres	Buy (NeonDB free tier → paid)	Branching DB is godsend for migrations
Realtime	Pusher / Ably	python-socketio (today)	Build	We have it; works fine; don't pay for fanout we already do
Vector embeddings	OpenAI / Cohere / Gemini	self-host BGE / E5	Buy (Gemini text-embedding-004)	Free tier covers v1; revisit if rate-limited

Decision	Buy	Build	Verdict	Why
OCR	Google Document AI (\$1.50/1k pages)	Tesseract + Gemini Vision	Buy (Gemini Vision)	Already have key, vision quality > Tesseract for Indian scripts
Court data ingestion	Manupatra/SCC partnership	Scrape ourselves	Build (open data) + Buy (Indian Kanoon API for fresh)	Manupatra B2B partnership is a multi-month sales cycle; we ship without it
Monitoring	Sentry + Datadog	Loki + Grafana	Buy (Sentry free tier; Datadog if we ever need APM)	Free tier is enough for v1
Customer support	Intercom / Crisp	DIY	Buy (Crisp free)	Lawyer has a question? They ask in chat. Don't reinvent

16. 12-week roadmap with kill criteria

Phase 1 (week 1-3) — Make Indian advocates love it

Goal: an Indian district lawyer searches the corpus, gets a real answer, drafts a notice. Today they can't. After this phase they can.

- **Ingest 100K+ Indian HC judgments** via india_vanga_hc (AWS S3 sync). Acceptance: searching "anticipatory bail Bombay HC 2023" returns ≥ 10 real cases with citations.

- **Streaming chat** (Gemini SSE $\rightarrow$ frontend EventSource). Acceptance: perceived latency drops from 8s to <2s.

- **WhatsApp chat export ingestion.** Acceptance: lawyer uploads _chat.txt, asks "summarise this conversation", gets a structured summary with timestamps.

- **Per-matter context** — Vault matters become first-class; every Assistant turn within a matter sees the full Vault.

- **Streaming "thinking" panel uses real signals** (BM25 hit count, retrieval source, validation status) instead of timer-based phases.

Kill criteria: if after 3 weeks (a) we don't hit 50 daily-active advocates, (b) average answer-grounded ratio doesn't lift to >75%, or (c) drafting still feels generic — pause and re-scope.

Phase 2 (week 4-6) — Audio + image + semantic recall

- **Audio transcription** via Sarvam Saaras (Indian) + Whisper (EN).
- **Image OCR** via Gemini Vision (orders, FIRs, IDs).
- **Hybrid retrieval** — BM25 + Gemini embeddings via Qdrant Cloud.

RRF fusion (k=60).

- **Citation verifier** as a separate FastAPI service —

/internal/verify-citation?id=... returns canonical metadata or

404. Every answer runs through it.

Kill criteria: if hybrid retrieval doesn't lift answer quality (tested via blind A/B on 50 advocate-rated answers) by $\geq 15\%$, fall back to BM25-only.

Phase 3 (week 7-9) — Compliance, multi-tenant, billing

- **Postgres migration** via existing `db_adapter.py` — flip

`DATABASE_URL` to NeonDB.

- **Multi-tenant** — every row gets `org_id`. Existing single-org users migrate to org #1.

- **RBAC** — Firm Admin / Partner / Associate / Paralegal / Client.

- **Stripe billing** — Free / Practitioner ■2K-mo / Firm

■5K-mo per seat.

- **Audit log export** — GET `/api/matters/:id/audit.pdf`.

Kill criteria: if migrating an existing user-base of ≥ 20 advocates breaks any flow, halt and patch before continuing.

Phase 4 (week 10-12) — Voice / WhatsApp surface (NyayaSathi funnel)

The full call surface from `CALL_SURFACE_PLAN.md`:

- Inbound voice (Twilio India DID).
- Inbound WhatsApp voice note + text.
- Outbound call-back from the chat.
- NyayaSathi → Sanhita handoff endpoint with phone-as-join-key.

Kill criteria: if Twilio voice quality on Indian mobile networks is unacceptable ($> 2s$ latency, dropped calls) — fall back to WhatsApp-only.

Phase 5+ (week 13+) — Predictive analytics, contract review, on-prem

- Predictive analytics framed as "comparable case settlement ranges"

(§10).

- Contract Review workflow (§12).
- On-prem deployment kit for top-100 firms.
- Mobile app (React Native or PWA — kill criteria: PWA install rate

$> 30\%$ means we don't need native).

17. What I'd change in the existing code

Specific, file-level. No nice-to-haves.

File	Change	Why
brief_service.py	Move <code>answer_open</code> to use Library + Vault retrieval (currently skips)	Drafting today is generic because we ignore the lawyer's own context
brief_service.py	Add <code>prefer_provider</code> and <code>cache_key</code> plumbing to every call	Context caching saves 75% on input tokens
validators/answer_gates.py	Soften G4 (grounding ratio) for explanation intents; keep G5 strict	Today refuses "tell me about X" too aggressively (we saw this in dev)
validators/input_guards.py	Redact PII in tool args before any logger call, not just before storage	DPDP compliance
connectors.py	Move web-search to a background fetch with cached results, not blocking on the chat path	A 5s Tavily fetch shouldn't make the user wait
realtime.py	Add per-IP connection cap; emit <code>bm25:reloaded</code> , <code>matter:opened</code> , <code>draft:saved</code> events	Future surfaces (live collaboration) need these
server.py	Move <code>/api/brief/chat</code> to <code>EventSourceResponse</code> for streaming	The single highest UX win
server.py	Add <code>/api/matters/*</code> (CRUD over a new <code>matters</code> table)	"Per-matter context" is foundational to Phase 1
auth.py	Add <code>org_id</code> column to every table; <code>users</code> becomes part of an <code>orgs</code> row	Multi-tenant; do it once before there's data to migrate
db_adapter.py	Add a connection pool warmup hook; lazy-init is too slow on first request after deploy	Cold-start UX
retrieval_pkg/index.py	Add <code>add_streaming(docs)</code> that doesn't refit BM25 each call	Today large ingests are $O(N^2)$ on the refit
web/src/app/app/page.tsx	Replace <code>localStorage</code> <code>model</code> persistence with the user-row preferences API	UX: settings should follow the user across devices
web/src/app/app/court-search-pane.tsx	Add citation graph view (cases that cite this case, cases this case cites)	The killer differentiator over Indian Kanoon
scripts/ingest_github_data.py	Add resume-from-checkpoint for the 80K+ Indian ingests	Today a network blip kills the run

18. New features ordered by ROI

#	Feature	Effort (eng-weeks)	User value	Strategic value	Verdict
1	Streaming chat (SSE)	0.5	10/10	8/10	Ship this week
2	India HC corpus 100K ingest	1	10/10	10/10	Ship this month

#	Feature	Effort (eng-weeks)	User value	Strategic value	Verdict
3	WhatsApp chat ingest	0.5	9/10	10/10 (moat)	Ship this month
4	Per-matter context	1	9/10	8/10	Ship this month
5	Audio transcription (Sarvam)	1	8/10	9/10	Phase 2
6	Image OCR (Gemini Vision)	0.5	7/10	8/10	Phase 2
7	Hybrid retrieval (BM25+vector)	2	7/10	8/10	Phase 2
8	Citation verifier service	1	8/10 (trust)	10/10 (moat)	Phase 2
9	Multi-tenant + Stripe	2	5/10 (until firm tier)	10/10 (revenue)	Phase 3
10	Audit log export PDF	0.5	8/10 (firms)	9/10 (compliance)	Phase 3
11	Voice + WhatsApp surface	3	9/10	10/10 (NyayaSathi funnel)	Phase 4
12	Citation graph view	1	8/10	9/10 (differentiator)	Phase 5
13	Contract Review workflow	2	8/10 (commercial work)	9/10 (high-margin)	Phase 5
14	Comparable-case settlement ranges	2	7/10	8/10	Phase 5
15	On-prem deployment kit	4	10/10 (top firms)	10/10 (■50L deals)	Phase 6
16	Mobile PWA polish	1	6/10	7/10	Phase 6
17	Native iOS/Android	8	7/10	6/10	Defer (PWA covers 80%)
18	Open API for partners	2	5/10	9/10 (ecosystem)	Phase 7

Read this table as: items 1-4 are urgent and decisive. Everything else is a phase-gate question — ship 1-4, see what breaks, then prioritise 5-8 based on user feedback.

Closing — the one decision that matters

Sanhita today is a complete *shape* with the right architecture. The single decision that determines whether it becomes a real business or a portfolio piece is:

Do we have 100,000 Indian judgments indexed by end of month?

If yes, every other roadmap item compounds. If no, every other roadmap item is moot.

Everything else — multi-modal, predictive, on-prem, voice — is amplifier. The base signal is the corpus.

— 2026-04-27, Sanhita v2 strategy doc.

Sanhita — Call surface plan

*Goal: let an Indian advocate query Sanhita by **phone call or WhatsApp voice note**, in their own language, and get back a grounded legal memo by SMS / WhatsApp / call-back. Same answers, same citations, same Gemini brain — just a non-browser entry point.*

This doc is self-contained for a fresh Claude session. Read it, then read SANHITA.md for the existing app surface.

What "the call part" means

Three distinct surfaces to build, in priority order:

1. **Inbound voice call** — advocate dials a Twilio Indian DID, speaks

their question (Hindi / Tamil / English / etc.), gets the answer read back by TTS in the same language, with the full memo + cites sent via SMS / WhatsApp afterwards.

2. **Inbound WhatsApp voice note** — same model but async: lawyer

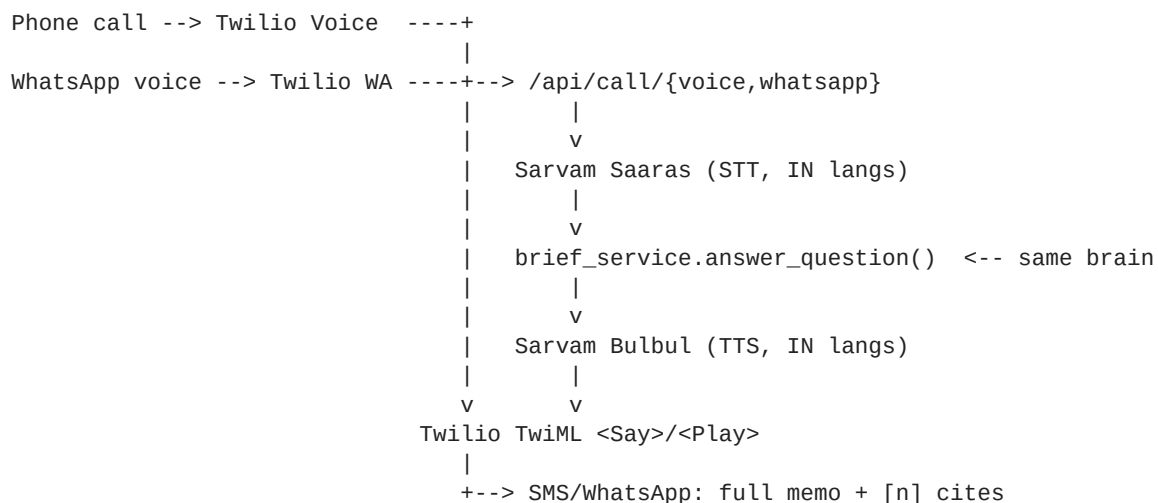
sends a WhatsApp voice note, Sanhita transcribes (Sarvam Saaras), answers, replies with the audio answer + a text memo.

3. **Outbound call-back from the web app** — from the Assistant pane,

click "Call me with the answer" → Twilio places an outbound call reading the latest answer aloud in the chosen language. Useful for advocates driving to court who already have a thread open.

All three reuse the **existing** `brief_service.answer_question / answer_open pipeline` — no new LLM, no new validator. Just adapters.

Architecture



What to build

Phase A — Twilio inbound voice (~3 days)

New file: `call_service.py`

- `handle_incoming_call(from_number, language_hint) -> TwiML`
- Greet caller in Hindi by default ("Sanhita mein swagat hai. Apna sawal poochiye.")
- `<Gather input="speech" speechTimeout="auto" language="hi-IN" action="/api/call/voice/answer">`
- `handle_voice_answer(speech_result, from_number) -> TwiML`
- Light-weight intent detection (chit-chat vs research) using the existing `_is_chitchat / _detect_intent` from `server.py`.
- Call `brief_service.answer_question(speech_result, hits=[], language=detected_lang) - hits=[]` lets the BM25 path fire if enabled, otherwise the model just talks.
- Truncate the answer to the first 2 sentences for `<Say>`. Send the full memo over SMS / WhatsApp follow-up.
- Persist as a row in a new `call_transcripts` SQLite table so the Dashboard's Activity feed shows "+91-9876543210 asked about anticipatory bail".

New endpoints (server.py):

- `POST /api/call/voice` — Twilio Voice webhook
- `POST /api/call/voice/answer` — speech-result handler
- `POST /api/call/voice/sms-followup` — sends the full memo as SMS

TwiML helpers — twilio SDK is already on PyPI. Add `twilio>=9.3.0` to `requirements.txt`.

Twilio config:

- Buy an Indian DID (~\$1/mo) at console.twilio.com.
- Set the Voice URL to `https://api.sanhita.ai/api/call/voice`.
- Enable speech recognition for Hindi, Tamil, Telugu, Bengali, English (multi-lang Gather is supported in Twilio).

Smoke test:

- Buy a \$1 Indian Twilio DID.
- Call it from your phone.
- Speak: "Section 138 ki limitation kya hai?"
- Expect: Hindi voice answer "138 ke liye limitation 30 din hain notice ke baad..." + SMS with full memo + cites.

Phase B — WhatsApp voice note (~2 days)

Twilio WhatsApp Sandbox is free; production needs a BSP.

- New endpoint: `POST /api/call/whatsapp` — Twilio WA webhook.
- If message contains a `MediaUrl`, fetch the audio, run through

Sarvam Saaras STT, then `answer_question`, then send back two WhatsApp messages: (1) Sarvam Bulbul TTS audio file, (2) text memo with cites.

- If text-only message, skip STT, go straight to answer.

Sarvam wiring:

- `llm/sarvam.py` already has `translate()`. Add `stt(audio_bytes, language) -> str` and `tts(text, language) -> bytes` helpers using the same `mayura:v1` API surface.

Phase C — Outbound call-back from web (~1 day)

Frontend:

- In `ChatBubble` action row (next to Copy / Email / Save as Doc),

add a "Call me" button that opens a small modal asking for a phone number + language.

Backend:

- `POST /api/call/outbound` — `{phone, message_id, language}` →

`twilio.client.calls.create(to=phone, from_=DID, url=...)` where the URL hosts a TwiML that `<Say>`s the answer in the chosen language.

Cost guard: rate-limit to 3 outbound calls per user per hour.

Files to create / modify

File	What
NEW <code>call_service.py</code>	TwiML builders + STT/TTS bridges
NEW <code>llm/sarvam.py</code> (extend)	Add <code>stt()</code> and <code>tts()</code> helpers
<code>server.py</code>	Add <code>/api/call/{voice, voice/answer, whatsapp, outbound}</code> endpoints
<code>auth.py</code>	Add <code>call_transcripts</code> table
<code>requirements.txt</code>	Add <code>twilio>=9.3.0</code>
<code>.env.example</code>	Add <code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_VOICE_DID</code> , <code>TWILIO_WA_FROM</code>
<code>web/src/app/app/page.tsx</code>	"Call me" button in <code>ChatBubble</code> action row

Existing code to reuse

- `brief_service.answer_question()` — drop in directly.
- `_is_chitchat()` and `_detect_intent()` from `server.py`.
- `validators/input_guards.py` — same length / scope / PII guards.
- `llm/sarvam.py translate()` — pattern to follow for `stt()`/`tts()`.
- `realtime.broadcast()` — fan out a `call:answered` event so the

Dashboard shows live call activity alongside web admin actions.

Costs

Item	Cost
Twilio India voice (inbound)	\$0.014/min
Twilio India SMS	\$0.0075/SMS
Twilio India DID	~\$1/mo
WhatsApp Business API	\$0.0065/conversation
Sarvam Saaras (STT)	usage-priced (~\$0.001/sec)
Sarvam Bulbul (TTS)	usage-priced
Gemini Flash answer	~\$0.001/answer

Per call (90 sec, ~30 sec speech, 60 sec TTS playback, 1 SMS):

- Voice: $\$0.014 \times 1.5 = \0.021
- STT: $\$0.001 \times 30 = \0.030
- TTS: ~\$0.005
- Gemini: \$0.001
- SMS: \$0.0075
- **~\$0.065 per call**

At ■5 (~\$0.06) revenue per call (typical Indian micro-payment), the unit economics work if 90%+ of calls convert to a satisfactory answer.

Smoke checklist

- [] Twilio Indian DID purchased and pointed at `/api/call/voice`.
- [] `TWILIO_*` env vars set in Railway / Render.
- [] `requirements.txt` has `twilio>=9.3.0`; `pip install clean`.
- [] `server.py` route `POST /api/call/voice` returns valid TwiML

(use `curl -X POST` with form data shaped like Twilio's webhook).

- [] End-to-end: call the DID, ask a §138 NI Act question, hear an answer in Hindi, receive an SMS with the full memo.
- [] Dashboard's Activity feed shows the call event live (Socket.io `call:answered` broadcast).

Out of scope for this work-stream

- Native iOS/Android app (Phase 7+).
- IVR menus / multi-step voice flows (just a single Q→A loop).
- Voice-to-voice conversation memory across calls (each call is its own thread; tie to a user via `from_number` lookup).
- Compliance: TRAI DLT for SMS in India is required for outbound SMS to non-Twilio-customers. Inbound is fine; outbound needs a header registered with one of the operator panels (Airtel/Jio/Vi).